Using any AI on this project (ChatGPT, Gemini, Copilot, Claude…)

Claude Code reads the project's rules by itself (`code_lecole/CLAUDE.md`, its hooks and the `/next-feature`, `/explain` commands). **Any other AI knows nothing about the project** until you give it the files. This guide says which files, in which order, and gives you prompts to copy and paste.

1. The files to give the AI

The easy way: upload your one pack: `team/pdf/AI-PACK-<your name>.pdf`. It holds, in this order:

#	Part	What the AI learns from it
1	The rules (`code_lecole/CLAUDE.md`)	How every line must be written — the AI must follow these
2	How it works (`team/HOW-IT-WORKS.md`)	Folders, one request through every file, security, white-label, the database
3	Why (`team/WHY.md`)	The reason behind each rule
4	Your tasks (`team/handover/<name>.md`)	What you build, in order, which pages and files
5	The decided details (`team/CHECKLIST.md`, "Backends left to connect")	Every rule already agreed for the tasks left, and who is told what

The AI can't see the code. When it asks for a file (or before it plans), paste the real files from `code_lecole/` — the controller, model, view, config it names. Never let it guess what a file contains.

2. The prompts, in order

Copy each prompt, fill in the `<…>`, and send it. Keep one chat per task.

Prompt 1 — learn the project (once per new chat)

```
I'm working on a team school-portal project: vanilla PHP 8, HTML, CSS and JS, no frameworks.
I've attached our project pack. Read all of it, in order: the rules, how it works, why, my tasks,
and the decided details. Don't write any code yet.

Then answer briefly:
1. How one request travels through the code (which folders and files, in order).
2. The 10 most important rules I must follow (security, where SQL goes, how forms are read,
   where school rules and permissions live, white-label, formatting).
3. What my first task is and which existing pages and files it touches.

From now on, follow these rules in every answer. If something isn't in the pack, ask me to
paste the file — never guess what a file contains.
```

Prompt 2 — plan a task (before any code)

```
Plan task <B-number and name> — no code yet.
```

```
Here are the files it touches (pasted below): <the page view, its components, the controller,
the model, the config, the mock data file in app/Mock/ — paste each with its path>.

Give me ONE complete plan, as tables:
1. Where it shows: page · component · what changes.
2. Rules and logic: each rule as "who may do what, when", and where it lives
   (config/permissions.php, config/white_label/rules.php, or the feature's config).
   Anything not decided in the pack → a question for the lead, with your recommended answer.
3. The database: the new update file database/updates/NNN_name.sql (the next free number),
   tables and columns.
4. The steps, small, each one commit: step · files · commit message "type(scope): …".
5. Tests: one curl for each refusal (no CSRF token, field[]=x, wrong role, signed out),
   and which pages to open in the browser for each role.
6. What gets deleted (the example data in app/Mock/, anything left unused).

Check the plan against every rule in the pack before you send it.
```

Send the plan to the lead before building.

Prompt 3 — build one step

```
Build step <number> of the plan: <the step>.
Here are the current files it changes: <paste them with their paths>.

Give me the COMPLETE new version of each file you change (not a fragment), with its path.
Follow the rules: SQL only in app/Models/ with prepared statements; form values only through
postText() / postPassword() / postList(); e() on every printed value; who may do what through
can() and config/permissions.php; numbers through rule() or the feature's config; school words
through word() / plural(); no inline style="", no !important; PHP and JS { on the next line,
CSS one property per line. Then explain each change in one short line.
```

After each step: read every line, run `bash bin/check.sh` (0 errors), open the page, commit.

Prompt 4 — review before the pull request

```
Review my changes against the project's rules. Here is the diff (git diff dev...HEAD):
<paste the diff>

List problems, most serious first, as: file:line — problem — fix.
Check: security (e(), CSRF only in core/Router.php, prepared statements, postText(), refuse() for
JSON, the same message for unknown and real accounts), rules in config (permissions.php, rules.php),
dead code, duplicates, white-label words, formatting, and that the example data it replaces is gone.
Say plainly if you find nothing. Only report what you actually see in the diff.
```

Prompt 5 — explain a file (until you can explain it yourself)

```
Explain <file path> so I can defend it in a viva. Here is the file and the files it calls:
<paste them>

1. One line: what the file is for.
2. The path of a request as a chain (routes/web.php → Controller::method() → Model::method() →
   view), saying what is passed at each arrow.
3. The code in small blocks: each block, then one short line per important line.
4. The rules it follows and why each matters.
5. Three likely viva questions with one-line answers.
```

Prompt 6 — practise for the code check

```
Give me 3 code-check exercises on my code (pasted below), like an examiner would ask:
one UI change (e.g. "show X on this card") and two business rules (e.g. "a student must be under 30",
"at most 3 per student"). For each: don't give the answer yet. When I send my answer, check it
against the project's rules and show me the files and lines that should change.
```

3. What you check yourself (no AI can do this for you)

- `bash bin/check.sh` inside `code_lecole/` → **0 errors** (it is also run on GitHub for every pull request).
- `php -l <file>` on every PHP file you changed.
- Open every page you touched, as every role that sees it — it must look exactly as before.
- The curl attacks from `team/ONBOARDING.md` → refused, data unchanged.
- `git diff` before every commit: you can explain every line.

4. Throw the answer away if the AI…

- puts SQL in a controller or a view, or glues values into SQL strings;
- reads `$_POST` directly, or checks the CSRF token by hand;
- types a role list (`['admin', 'management']`), a number (`30`, `300`) or the school's name into a controller;
- adds a library, a framework, inline `style=""`, `!important` or a new colour;
- changes files outside your task, or how a page looks;
- gives you a fragment ("… rest unchanged") instead of the whole file;
- invents a file, a function or a column it never saw.